Now let's use a simple example to show how to deploy a Flask app on Heroku.

The requirements are:

- Python
- Pip
- Heroku CLI
- Git

Basic knowledge of Git is important as we use a lot of its command lines. Nevertheless, just follow the instructions given below.

Step 1: Download Heroku CLI

The Heroku CLI is used for managing the application after deployment. After downloading the CLI, run the following command in the terminal to log in:

```
heroku login
```

Make sure you use the credentials that you entered while signing up on the Heroku platform, i.e., email id and password.

Step 2: Creating the app

Create a folder and give it a name, e.g., *firstflask*. Inside the folder, open a command window and start on the creation of a simple Flask application.

i. Create a virtual environment with *pipenv* and then install Flask and Gunicorn. Type and run the following command:

```
$ pipenv install flask gunicorn
```

ii. Create a *Procfile*. It is used to hold commands that will run on startup. Then write the following code in it:

```
$ touch Procfile
```

Figure 1: Procfile

iii. Next, create *runtime.txt* and then write the following code:

```
$ touch runtime.txt
```

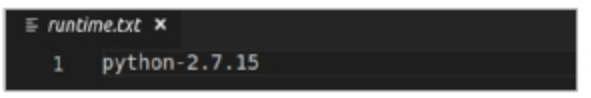

Figure 2: Runtime

iv. Create a folder and give it a name of your choice, e.g., *app*. Then access the folder:

```
$ mkdir app
$ cd app
```

v. Next, create a Python file, i.e., *main.py:*

```
touch main.py
```

vi. Write the following code in *main.py:*

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home_view():
        return "<h1>Welcome to Your
first App on Heroku</h1>"
```

vii. In the already created *firstflask* directory, create a *wsgi.py* file and write the following code in it:

```
$ cd ../
$ touch wsgi.py
```

viii. Write the following code in the *wsgi.py* file:

```
    filter_none
brightness_4
from app.main import app
    if __name__ == "__main__":
        app.run()
```

ix. Run the created virtual environment:

```
$ pipenv shell
```

Step 3: Deploying the app

i. In Git, initialise an empty repo and add files in it, and then commit the changes.

```
$ git init
$ git add .
$ git commit -m "Initial Commit"
```

ii. Log in to Heroku CLI:

```
heroku login
```

iii. Create a preferred name for your Web application, say, *firstflask:*

```
$ heroku create eflask-app
```

iv. Run the code:

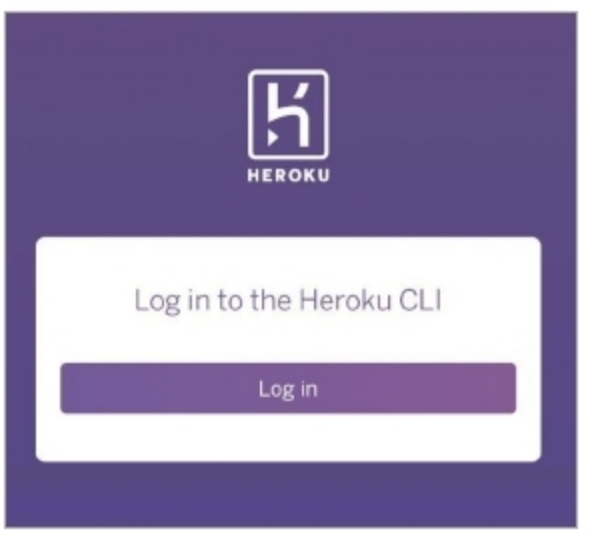

Figure 3: Heroku login

```
$ git push heroku master
```

And that's it. You have just uploaded your first app to the Heroku cloud using Flask. Congratulations! Needless to say, you are just getting started here. There is much more that can be done using the Heroku platform. I hope this article motivates you to get started with writing your apps to the Heroku platform and joining the cloud revolution. END

By: Mir H.S. Quadri

The author is a post-graduate scholar and researcher in the field of AI/ML. He has published papers in reputed scientific journals. He is also a FOSS enthusiast, and actively contributes to several open source projects. He runs the *blogcodelatte.site*, where he shares valuable insights and tutorials on emerging technologies.